Relatório Prático: Comparação entre Árvore AVL e Árvore Rubro-Negra em um Sistema Bancário

Fran Robson de Souza[1], **Jenner Lucas Carneiro**[1], **Raul Berger**[1], **Severino de Souza**[1]

[1]Instituto de Ciências Exatas e Tecnologia (ICET)
Universidade Federal do Amazonas (UFAM)
Itacoatiara – AM – Brasil

Abstract. *This report presents an experimental and practical comparison between AVL and Red-Black trees implemented from scratch in Python for a banking system (Minimundo 6: Bank Account). We evaluated tree height, rotation counts, recolorations, and runtime performance across datasets ranging from 100 to 10,000 accounts. The results show that while the AVL tree maintains a strictly shorter tree under sequential insertions, the Red-Black tree is noticeably faster in insertion and deletion operations due to its ability to resolve most balance conflicts through quick recolorations rather than heavy pointer restructuring.*

Resumo. *Este relatório apresenta um comparativo prático e direto entre as árvores AVL e Rubro-Negra, implementadas do zero em Python para gerenciar contas bancárias (Minimundo 6). Avaliamos o comportamento das duas estruturas com conjuntos de 100 até 10.000 contas, medindo altura final, número de rotações, recolorações, quantidade de nós visitados em buscas e tempo de execução. Os resultados mostram que, embora a AVL mantenha a árvore mais baixa quando os dados entram ordenados, a Rubro-Negra foi mais rápida para inserir e remover na maioria dos testes, já que ela gasta menos tempo mexendo em ponteiros e resolve boa parte dos desequilíbrios apenas trocando as cores dos nós.*

1. Introdução

Quando precisamos guardar e buscar dados rapidamente na memória, as árvores binárias de busca (ABB) comuns são uma das primeiras opções que vêm à mente. O problema é que, se inserirmos dados já em ordem crescente, a árvore vira uma lista encadeada e perde toda a sua eficiência.

Para resolver isso, existem as árvores auto-balanceáveis. Neste trabalho prático da disciplina de Algoritmos e Estruturas de Dados II (UFAM/ICET), implementamos e comparamos duas das principais árvores da literatura: a Árvore AVL e a Árvore Rubro-Negra. Aplicamos as duas estruturas no Minimundo 6: Sistema Bancário, onde cada nó representa uma conta bancária.

Nosso objetivo foi testar na prática como cada árvore se comporta na inserção em ordem crescente, em ordem aleatória, nas buscas por contas e nas remoções de clientes, verificando se o esforço de manter a árvore mais equilibrada realmente compensa no tempo final de execução.

2. Descrição da Solução

2.1. O Registro de Conta Bancária

Cada conta cadastrada no sistema possui os campos definidos no edital: número da conta (número inteiro único entre 5 e 6 dígitos, de 10001 a 999999, que serve como chave primária e define a posição do nó na árvore), titular (nome do cliente), agência (código fixo 0001), tipo de conta (Corrente, Poupança ou Salário) e saldo inicial em reais, que atua apenas como dado do cliente sem influenciar a topologia da árvore.

Seguindo a regra da Seção 8 do edital, o sistema não aceita contas repetidas. Se o usuário ou o teste tentar inserir um número de conta que já existe, a operação é rejeitada de forma segura.

2.2. A Árvore AVL

Na nossa AVL, cada nó armazena sua própria altura. O equilíbrio é controlado pelo Fator de Balanceamento (FB):

$$FB = altura(filhoesquerdo) - altura(filhodireito) \tag{1}$$

Consideramos que nós nulos têm altura -1 e folhas têm altura 0. Sempre que uma inserção ou remoção deixa algum nó com $FB = 2$ ou $FB = -2$, aplicamos as rotações necessárias: simples à direita (caso LL), simples à esquerda (caso RR), dupla esquerda-direita (caso LR) ou dupla direita-esquerda (caso RL).

Na remoção, quando o nó a ser excluído tem dois filhos, localizamos o sucessor em-ordem (o menor nó da subárvore direita), copiamos os dados dele para a posição atual e removemos o sucessor da subárvore direita, recalculando as alturas de baixo para cima até a raiz.

2.3. A Árvore Rubro-Negra

A nossa Rubro-Negra segue as regras do livro do Cormen (capítulo 13). Utilizamos um único nó sentinela compartilhado chamado `NIL`, que tem cor preta e representa todas as folhas vazias e o pai da raiz. Isso deixou o código mais limpo e evitou checagens de ponteiro nulo.

As regras mantidas são:

1. Todo nó é vermelho ou preto.
2. A raiz é sempre preta.
3. Todo `NIL` é preto.
4. Nó vermelho não pode ter filho vermelho.
5. Todos os caminhos da raiz até as folhas `NIL` têm exatamente a mesma quantidade de nós pretos (altura negra).

Na remoção de nós com dois filhos, o sucessor em-ordem vai para o lugar do nó removido e herda a cor dele. Se o nó que saiu da árvore era vermelho, a altura negra continua igual e nenhuma rotação é feita. Se o nó que saiu era preto, a árvore perde um preto naquele caminho e chamamos o método de correção (`_remover_fixup`), que resolve os casos de 1 a 4 com rotações e trocas de cor.

3. Metodologia dos Experimentos

Para que os resultados fossem justos e reproduzíveis por qualquer pessoa que execute o código, usamos os seguintes critérios:

- Mesmos dados: sempre que um teste rodava, tanto a AVL quanto a Rubro-Negra recebiam exatamente a mesma sequência de contas. Na remoção, o subconjunto de chaves a remover é sorteado uma única vez por semente determinística e aplicado de forma idêntica em ambas as árvores, garantindo paridade absoluta de nós excluídos.
- Sementes fixas: usamos a semente 42 (`seed = 42`) para gerar as contas aleatórias, semente 142 para as buscas e 242 para as remoções.
- Tamanhos testados: rodamos os tamanhos padrão do edital ($N = 100$, 1.000 e 10.000) e também adicionamos testes da equipe com $N = 500$ e $N = 5.000$ para enxergar melhor a curva de crescimento.
- Critério de busca (Experimento C): para cada N, fizemos N buscas. Metade das buscas (50%) foram por contas que sabíamos que estavam na árvore, e a outra metade (50%) por contas inexistentes, garantindo que a busca descesse até a base da árvore.
- Remoções parciais (Experimento D): testamos a remoção de 10%, 30% e 50% das contas cadastradas. Nas Tabelas 1 e 2 apresentamos a medição oficial de 30% (padrão da Seção 5.1 do edital), enquanto as taxas de 10% e 50% constam na planilha de dados brutos exportada.
- Medição de tempo: o tempo foi medido em milissegundos usando `time.perf_counter()`, mantendo as mensagens de tela desligadas para não deixar a medição lenta por causa do terminal.

4. Resultados Obtidos

Para responder a todas as exigências do edital e avaliar as duas árvores tanto no pior cenário teórico quanto no fluxo bancário real, executamos os experimentos sob duas ordens de entrada distintas:

- A Tabela 1 apresenta as métricas sob inserção em ordem crescente (Experimento A com buscas e remoções), simulando a carga sequencial contínua que representa o pior caso clássico de uma Árvore Binária de Busca.
- A Tabela 2 apresenta as métricas sob inserção em ordem aleatória (Experimentos B, C e D), simulando o fluxo normal e imprevisível de abertura de contas bancárias.

Tabela 1. Métricas Obrigatórias – Inserção em Ordem Crescente (Pior Caso de ABB)

N	Estrutura	Alt.	Inserção			Busca (50%/50%)		Remoção (30%)		
			Rot.	Recol.	Tempo (ms)	Nós Méd.	Tempo (ms)	Rot.	Recol.	Tempo (ms)
100	AVL	6	93	N/A	0.691	6.46	0.171	1	N/A	0.181
	Rubro-Negra	10	89	445	0.699	8.64	0.275	3	71	0.101
500	AVL	8	491	N/A	4.286	8.44	1.165	14	N/A	1.160
	Rubro-Negra	14	485	2.418	4.505	11.67	2.124	20	422	0.543
1.000	AVL	9	990	N/A	10.935	9.48	2.603	26	N/A	2.516
	Rubro-Negra	16	983	4.909	10.646	13.20	4.404	42	822	1.115
5.000	AVL	12	4.987	N/A	56.642	12.18	17.462	129	N/A	17.730
	Rubro-Negra	21	4.978	24.886	63.538	16.94	30.166	179	4.172	7.561
10.000	AVL	13	9.986	N/A	118.701	13.18	56.057	253	N/A	65.274
	Rubro-Negra	23	9.976	49.877	136.134	18.45	102.141	326	8.217	25.855

Tabela 2. Métricas Obrigatórias – Inserção em Ordem Aleatória (Cenário Típico Bancário)

N	Estrutura	Alt.	Inserção			Busca (50%/50%)		Remoção (30%)		
			Rot.	Recol.	Tempo (ms)	Nós Méd.	Tempo (ms)	Rot.	Recol.	Tempo (ms)
100	AVL	7	52	N/A	0.582	6.40	0.164	4	N/A	0.191
	Rubro-Negra	7	42	200	0.505	7.01	0.224	13	64	0.105
500	AVL	10	330	N/A	4.035	8.16	1.089	70	N/A	1.209
	Rubro-Negra	10	299	1.184	3.015	8.65	1.431	60	292	0.474
1.000	AVL	11	701	N/A	8.948	9.11	2.518	107	N/A	2.639
	Rubro-Negra	11	592	2.267	6.559	9.12	3.068	83	484	0.980
5.000	AVL	14	3.439	N/A	55.025	12.28	17.723	573	N/A	18.622
	Rubro-Negra	15	2.915	11.552	45.112	11.85	21.356	520	2.590	6.409
10.000	AVL	15	6.895	N/A	126.947	13.80	41.574	1.130	N/A	41.576
	Rubro-Negra	15	5.764	23.012	96.571	12.82	47.321	986	5.031	14.501

5. Análise Escrita dos Resultados

Abaixo respondemos às sete perguntas solicitadas na Seção 5.2 do edital, confrontando os resultados obtidos em ambas as tabelas:

5.1. Qual estrutura apresentou menor altura nos cenários testados?

Depende muito da ordem de entrada das contas:

- Na inserção crescente (Tabela 1): a AVL foi expressivamente superior no controle de profundidade. Para $N = 10.000$, a AVL terminou com altura 13, enquanto a Rubro-Negra atingiu altura 23. A AVL não tolera desnível superior a 1 entre subárvores e forçou a árvore a se manter perfeitamente compacta. A Rubro-Negra, por permitir sequências de nós pretos e vermelhos alternados, alongou seus ramos pela direita.
- Na inserção aleatória (Tabela 2): as duas estruturas apresentaram alturas praticamente idênticas. Com 100 contas, ambas terminaram com altura 7; com 1.000 contas, ambas com altura 11; e com 10.000 contas, ambas com altura 15. Em dados não-ordenados, o equilíbrio natural da inserção mantém ambas as árvores no mesmo patamar de compactação.

5.2. Qual estrutura realizou mais rotações durante inserções e remoções?

A Árvore AVL realizou mais rotações do que a Rubro-Negra nas cargas aleatórias, enquanto na inserção crescente houve quase um empate técnico em rotações.

Na Tabela 1 ($N = 10.000$ crescente), como cada nova chave entrava obrigatoriamente no extremo direito, ambas as árvores foram forçadas a girar quase uma vez por inserção: 9.986 rotações na AVL e 9.976 na Rubro-Negra.

Já na Tabela 2 ($N = 10.000$ aleatório), a AVL realizou 6.895 rotações de inserção contra 5.764 da Rubro-Negra (uma economia de 1.131 rotações a favor da Rubro-Negra). Na remoção de 30%, a AVL fez 1.130 rotações contra 986 da Rubro-Negra. A Rubro-Negra absorve muitos conflitos apenas alternando as cores de nós vermelhos e pretos.

Um detalhe que chama atenção na Tabela 2 ocorre para $N = 100$, onde a AVL fez apenas 4 rotações de remoção contra 13 da Rubro-Negra. Como o lote de 30 contas removidas foi exatamente o mesmo em ambas as árvores, essa divergência pontual se explica pelas regras de correção de cada estrutura: na Rubro-Negra, remover nós pretos gera o nó duplo-preto, que nos Casos 1 e 4 do Cormen força rotações imediatas; na AVL, muitas dessas mesmas exclusões ocorreram em folhas cujos pais mantiveram o fator de balanceamento dentro do limite $|FB| \leq 1$. No entanto, conforme o tamanho da base cresce ($N \geq 500$ até 10.000), a tendência assintótica prevalece e a AVL passa a realizar mais rotações em todas as operações de remoção.

5.3. Como as recolorações da Rubro-Negra aparecem nos experimentos?

As recolorações são a principal estratégia da Rubro-Negra para restaurar o equilíbrio com baixo custo computacional. Elas aparecem em volume expressivo em todos os testes:

- Na inserção crescente de 10.000 contas (Tabela 1), ocorreram impressionantes 49.877 recolorações na inserção e 8.217 na remoção.

- Na inserção aleatória de 10.000 contas (Tabela 2), ocorreram 23.012 recolorações na inserção e 5.031 na remoção.

Essas recolorações ocorrem principalmente no Caso 1 de inserção (quando o tio do nó recém-inserido é vermelho, bastando passar a cor preta ao pai e ao tio e vermelha ao avô) e nos casos de remoção onde o nó duplo-preto transfere peso aos ancestrais.

5.4. A menor altura resultou sempre em menor tempo de execução?

Não. A menor altura garantiu menos comparações na busca, mas não se traduziu em menor tempo para escrita e atualização.

Na Tabela 1 (crescente com $N = 10.000$), a menor altura da AVL (13 contra 23) refletiu-se diretamente no tempo de busca: a AVL precisou em média de apenas $13,18$ nós visitados por busca ($56,0ms$), enquanto a Rubro-Negra precisou de $18,45$ nós visitados ($102,1ms$).

Já na Tabela 2 (aleatório), mesmo tendo altura igual ou ligeiramente menor, a AVL foi mais lenta na inserção ($126,9ms$ contra $96,5ms$) e expressivamente mais lenta na remoção de 30% ($41,5ms$ contra $14,5ms$, onde a Rubro-Negra foi quase três vezes mais rápida). Cada rotação na AVL exige a atualização de múltiplos ponteiros em memória e a recomputação ascendente das alturas. A Rubro-Negra resolve grande parte dos casos via recoloração, que é uma atribuição simples de variável em memória.

5.5. Como a ordem de inserção influenciou as duas estruturas?

A ordem de inserção impactou fortemente a topologia de ambas as árvores:

- Na ordem crescente, o desequilíbrio contínuo à direita forçou ambas as árvores ao seu limite de rebalanceamento. A AVL respondeu com reestruturações contínuas para manter altura estritamente mínima (13). A Rubro-Negra respondeu com 49.877 recolorações, priorizando a velocidade da escrita em detrimento da profundidade (23).
- Na ordem aleatória, os nós se distribuíram uniformemente, gerando árvores simétricas onde ambas as estruturas operaram próximas de suas alturas ótimas (≈ 15) com rotações moderadas.

5.6. Em quais operações as diferenças entre AVL e Rubro-Negra ficaram mais evidentes?

As maiores diferenças apareceram na remoção e na inserção em massa.

Na remoção, a Rubro-Negra foi muito superior no tempo de resposta (cerca de um terço do tempo da AVL). Isso acontece porque, se o nó que sai da árvore for vermelho, a altura negra nem muda e a Rubro-Negra não precisa fazer nada. Na AVL, toda remoção obriga a recalcular alturas e fatores de balanceamento subindo os ancestrais.

A outra diferença marcante foi na altura com entrada crescente, onde o rigor da AVL provou ser imbatível para segurar a profundidade da árvore.

5.7. Quais limitações metodológicas existem na comparação realizada?

Ao analisar os testes, devemos considerar alguns pontos:

- Usamos Python, que é uma linguagem interpretada e possui o coletor de lixo da memória (*garbage collection*), o que pode causar pequenas variações de milissegundos de uma rodada para outra.
- Em árvores com ponteiros na memória RAM, o processador pode sofrer com falhas de cache (*cache miss*) porque os nós ficam espalhados na memória.
- Testamos até 10.000 contas. Em um banco do mundo real com milhões de clientes, as contas ficam gravadas em disco ou banco de dados, onde estruturas como Árvores B e B+ costumam ser usadas no lugar de árvores binárias puras em memória.

6. Conclusão

O trabalho permitiu comprovar na prática a teoria vista em sala de aula. Cada árvore tem o seu ponto forte:

- A Árvore AVL é ideal para sistemas onde as contas raramente são inseridas ou apagadas, mas sofrem milhões de consultas por minuto. Por ser mais rigorosa na altura, o caminho até as contas é um pouco menor.
- A Árvore Rubro-Negra é muito superior para sistemas bancários dinâmicos do dia a dia, onde clientes abrem e fecham contas a todo momento. Por priorizar a recoloração e fazer menos rotações, ela gasta menos tempo de máquina e tem um desempenho de escrita muito mais veloz.

Referências

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: teoria e prática*. 2. ed. Rio de Janeiro: Campus, 2002.

[2] ADELSON-VELSKII, G. M.; LANDIS, E. M. An algorithm for the organization of information. *Soviet Mathematics Doklady*, v. 3, p. 1259-1263, 1962.

[3] BRITO, A. *Trabalho Prático I – Algoritmos e Estruturas de Dados II*. Universidade Federal do Amazonas (UFAM), ICET, 2026.